ST JOSEPH HOSPITAL

Patient Registration System

Complete Project: HTML, JavaScript Validation, Node.js Server and SQL Database

Prepared by: [Your Name]

College Project / Web Development

Project Overview

The St Joseph Hospital Patient Registration System lets a new patient fill an online registration form before the first consultation. The form is checked in the browser, checked again on the server and then saved in a SQL database. After saving, the patient sees a registration success page.

Technologies used

- Front end: HTML5 and CSS3 (registration form, login page, success page)
- Validation: JavaScript in the browser (validation.js)
- Back end: Node.js built-in http module
- Database: SQLite (SQL), table created with schema.sql

How it works

1. The patient opens the registration form (index.html).
2. validation.js checks every field and shows an error message under any wrong field.
3. When all checks pass, validation.js sends the data to the server as JSON with fetch().
4. server.js validates the data again and saves it in the patients table with a prepared INSERT statement.
5. A duplicate mobile number is rejected by the UNIQUE constraint in the database.
6. On success the browser opens registration-success.html.

Project folder

```
hospital-app/
  server.js          Node.js server + database code
  schema.sql         SQL table definition
  hospital.db        Database file (created automatically)
  public/
    index.html                  Patient registration form
    validation.js               Form validation + sends data to server
    registration-success.html   Page shown after saving
```

Steps to run

1. Install Node.js 22.13 or newer (no extra packages are needed).
2. Open a terminal inside the hospital-app folder.
3. Run: node server.js
4. Open http://localhost:3000 and submit the registration form.
5. View the saved rows: sqlite3 hospital.db "SELECT * FROM patients;"

Contents

- Output Screenshots: login page, patient registration page, registration successful page, database table
- Source Code: index.html, validation.js, server.js, schema.sql, registration-success.html

Output Screenshot 1 – Login Page

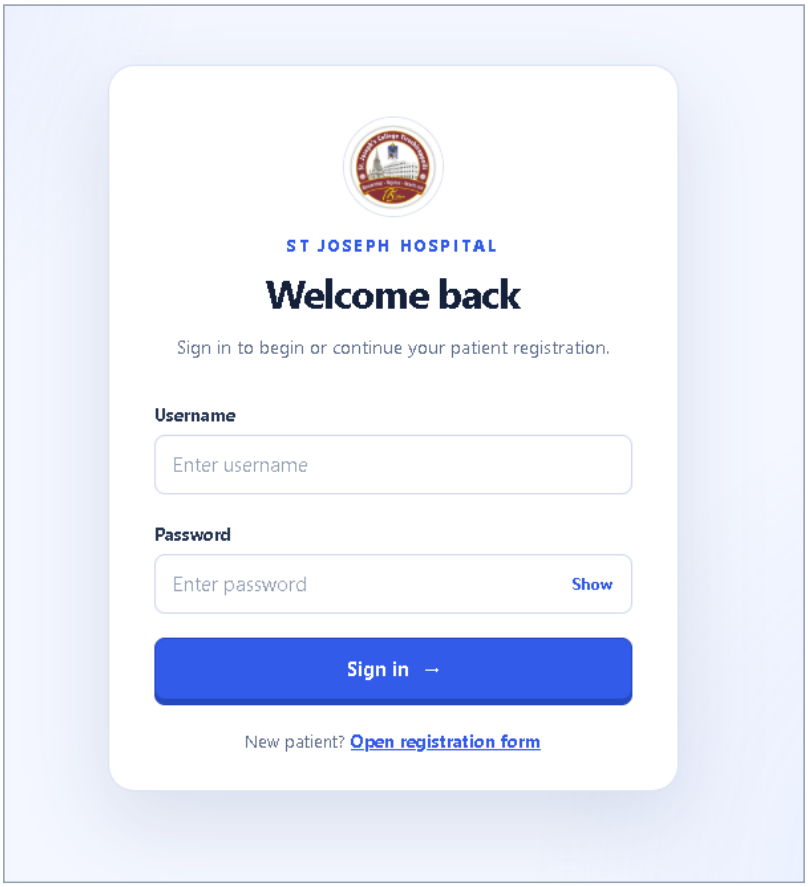


Figure 1: St Joseph Hospital login page displayed in the browser.

Output Screenshot 2 – Patient Registration Page

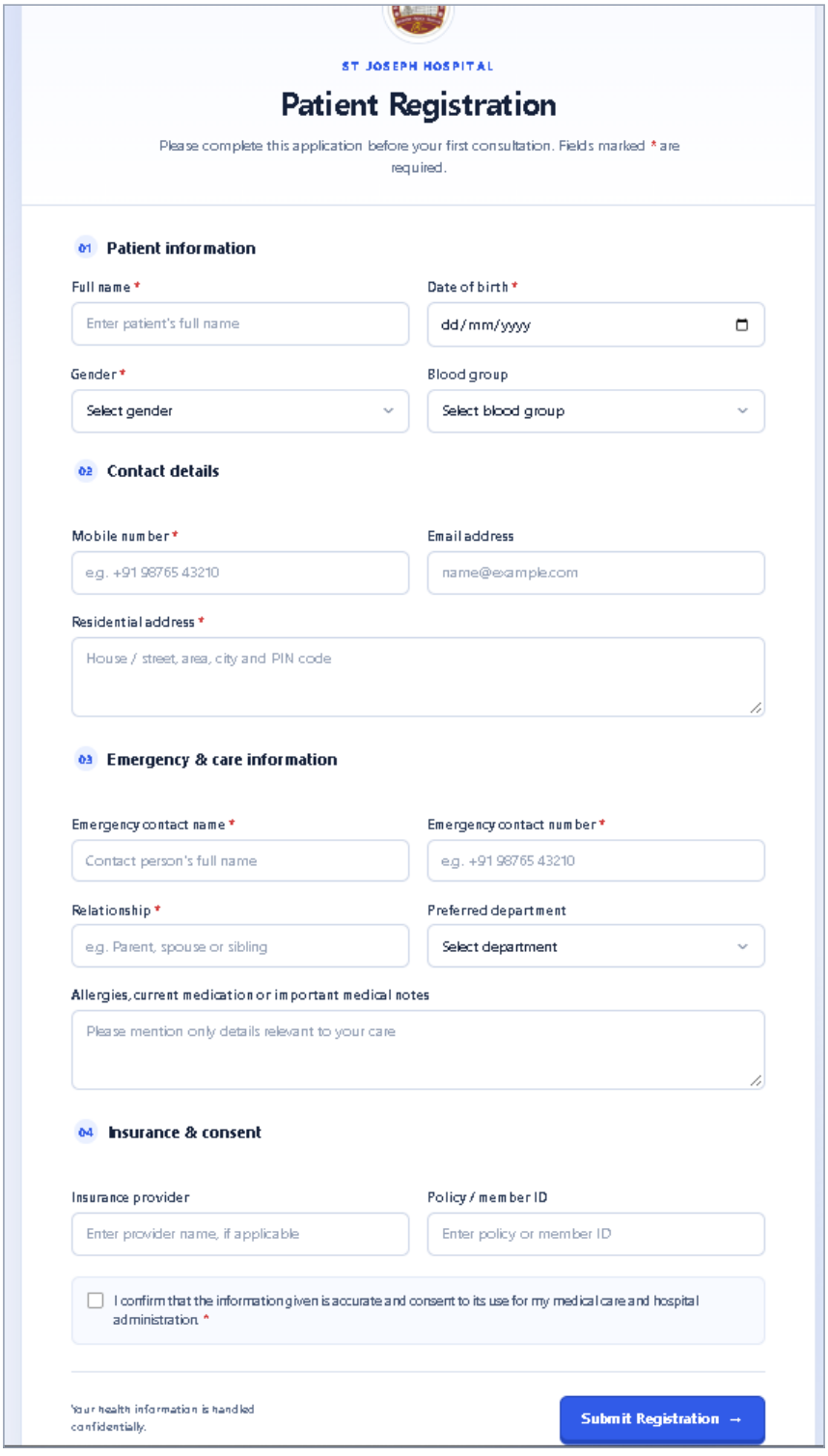


Figure 2: Patient registration form displayed in the browser.

Output Screenshot 3 – Registration Successful Page

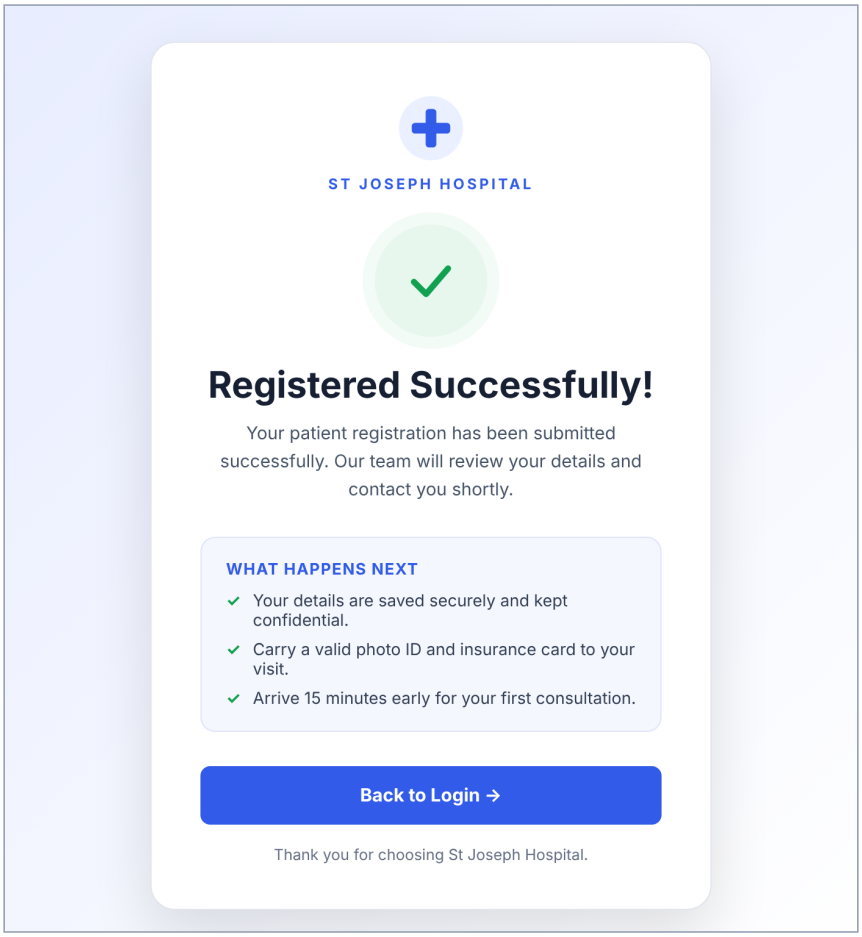
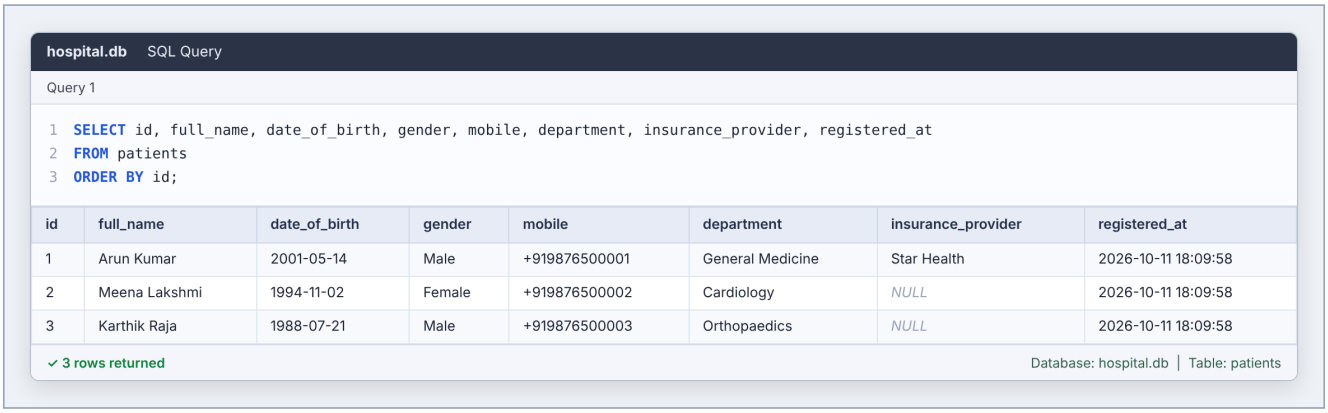


Figure 3: Page shown after the registration form passes validation and is saved.

Output Screenshot 4 – Database Table (SQL Output)



hospital.db SQL Query

Query 1

```
1 SELECT id, full_name, date_of_birth, gender, mobile, department, insurance_provider, registered_at
2 FROM patients
3 ORDER BY id;
```

id	full_name	date_of_birth	gender	mobile	department	insurance_provider	registered_at
1	Arun Kumar	2001-05-14	Male	+919876500001	General Medicine	Star Health	2026-10-11 18:09:58
2	Meena Lakshmi	1994-11-02	Female	+919876500002	Cardiology	*NULL*	2026-10-11 18:09:58
3	Karthik Raja	1988-07-21	Male	+919876500003	Orthopaedics	*NULL*	2026-10-11 18:09:58

✓ 3 rows returned — Database: hospital.db | Table: patients

Figure 4: Patient records saved in the patients table after submitting the registration form.

Sample test data was used for the three patient rows.

HTML Source Code – index.html

The patient registration form with patient information, contact details, emergency and care information, and insurance and consent sections. It loads validation.js at the end of the page.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Patient registration application form for St Joseph Hospital.">
  <title>St Joseph Hospital | Patient Registration</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main class="page-shell">
    <section class="registration-card" aria-labelledby="form-title">
      <header class="form-header">
        <div class="brand-mark" aria-hidden="true">
          <svg viewBox="0 0 64 64" role="img">
            <circle cx="32" cy="32" r="30" fill="#eaf0ff"/>
            <rect x="27" y="14" width="10" height="36" rx="3" fill="#315be8"/>
            <rect x="14" y="27" width="36" height="10" rx="3" fill="#315be8"/>
          </svg>
        </div>
        <p class="eyebrow">ST JOSEPH HOSPITAL</p>
        <h1 id="form-title">Patient Registration</h1>
        <p class="intro">Please complete this application before your first consultation. Fields marked <span
          aria-hidden="true">*</span> are required.</p>
      </header>

      <form class="registration-form" action="#" method="post">
        <fieldset>
          <legend><span>01</span> Patient information</legend>
          <div class="form-grid two-columns">
            <div class="field">
              <label for="full-name">Full name <b>*</b></label>
              <input id="full-name" name="full-name" type="text" autocomplete="name" placeholder="Enter patient's full
                name" required>
            </div>
            <div class="field">
              <label for="date-of-birth">Date of birth <b>*</b></label>
              <input id="date-of-birth" name="date-of-birth" type="date" required>
            </div>
            <div class="field">
              <label for="gender">Gender <b>*</b></label>
              <select id="gender" name="gender" required>
                <option value="" selected disabled>Select gender</option>
                <option>Female</option>
                <option>Male</option>
                <option>Non-binary</option>
                <option>Prefer not to say</option>
              </select>
            </div>
            <div class="field">
              <label for="blood-group">Blood group</label>
              <select id="blood-group" name="blood-group">
                <option value="" selected>Select blood group</option>
                <option>A positive (A+)</option>
                <option>A negative (A-)</option>
                <option>B positive (B+)</option>
                <option>B negative (B-)</option>
                <option>AB positive (AB+)</option>
                <option>AB negative (AB-)</option>
                <option>O positive (O+)</option>
                <option>O negative (O-)</option>
              </select>
            </div>
          </div>
        </fieldset>

        <fieldset>
          <legend><span>02</span> Contact details</legend>
          <div class="form-grid two-columns">
            <div class="field">
              <label for="mobile">Mobile number <b>*</b></label>
              <input id="mobile" name="mobile" type="tel" autocomplete="tel" inputmode="tel" placeholder="e.g. +91 98765
                43210" required>
            </div>
            <div class="field">
              <label for="email">Email address</label>
              <input id="email" name="email" type="email" autocomplete="email" placeholder="name@example.com">
            </div>
```

```
            <div class="field field-wide">
              <label for="address">Residential address <b>*</b></label>
              <textarea id="address" name="address" rows="3" autocomplete="street-address" placeholder="House / street,
                area, city and PIN code" required></textarea>
            </div>
          </div>
        </fieldset>

        <fieldset>
          <legend><span>03</span> Emergency &amp;amp; care information</legend>
          <div class="form-grid two-columns">
            <div class="field">
              <label for="emergency-name">Emergency contact name <b>*</b></label>
              <input id="emergency-name" name="emergency-name" type="text" placeholder="Contact person's full name"
                required>
            </div>
            <div class="field">
              <label for="emergency-phone">Emergency contact number <b>*</b></label>
              <input id="emergency-phone" name="emergency-phone" type="tel" inputmode="tel" placeholder="e.g. +91 98765
                43210" required>
            </div>
            <div class="field">
              <label for="relationship">Relationship <b>*</b></label>
              <input id="relationship" name="relationship" type="text" placeholder="e.g. Parent, spouse or sibling"
                required>
            </div>
            <div class="field">
              <label for="department">Preferred department</label>
              <select id="department" name="department">
                <option value="" selected>Select department</option>
                <option>General Medicine</option>
                <option>Cardiology</option>
                <option>Dermatology</option>
                <option>Orthopaedics</option>
                <option>Paediatrics</option>
                <option>Women’s Health</option>
                <option>Other / Not sure</option>
              </select>
            </div>
            <div class="field field-wide">
              <label for="medical-notes">Allergies, current medication or important medical notes</label>
              <textarea id="medical-notes" name="medical-notes" rows="3" placeholder="Please mention only details relevant
                to your care"></textarea>
            </div>
          </div>
        </fieldset>

        <fieldset>
          <legend><span>04</span> Insurance &amp;amp; consent</legend>
          <div class="form-grid two-columns">
            <div class="field">
              <label for="insurance-provider">Insurance provider</label>
              <input id="insurance-provider" name="insurance-provider" type="text" placeholder="Enter provider name, if
                applicable">
            </div>
            <div class="field">
              <label for="policy-number">Policy / member ID</label>
              <input id="policy-number" name="policy-number" type="text" placeholder="Enter policy or member ID">
            </div>
          </div>

          <label class="check-row" for="consent">
            <input id="consent" name="consent" type="checkbox" required>
            <span>I confirm that the information given is accurate and consent to its use for my medical care and hospital
              administration. <b>*</b></span>
          </label>
        </fieldset>

        <div class="form-footer">
          <p>Your health information is handled confidentially.</p>
          <button type="submit">Submit Registration <span aria-hidden="true">→</span></button>
        </div>
      </form>
    </section>
  </main>
  <script src="validation.js"></script>
</body>
</html>
```

JavaScript Source Code – validation.js

Checks every field of the registration form, shows error messages, and after all checks pass sends the data to the server with fetch() and opens the success page.

```
/*
 * St Joseph Hospital - Patient Registration Form Validation
 * Usage: add <script src="validation.js"></script> just before </body> in index.html
 */

document.addEventListener("DOMContentLoaded", function () {
  var form = document.querySelector(".registration-form");

  if (!form) {
    return;
  }

  // Use our own messages instead of the browser's default pop-ups
  form.setAttribute("novalidate", "");

  // Small style block for the error messages
  var style = document.createElement("style");
  style.textContent =
    ".error-message{display:block;margin-top:6px;font-size:12px;font-weight:600;color:#d92d20;}" +
    ".field-invalid{border-color:#d92d20 !important;box-shadow:0 0 0 3px rgba(217,45,32,0.12) !important;}";
  document.head.appendChild(style);

  var NAME_PATTERN = /^[A-Za-z][A-Za-z .'-]{2,59}$/;
  var PHONE_PATTERN = /^(?:\+91|91|0)?[6-9]\d{9}$/;
  var EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
  var POLICY_PATTERN = /^[A-Za-z0-9\/-]{4,30}$/;

  function value(id) {
    return document.getElementById(id).value.trim();
  }

  function cleanPhone(text) {
    return text.replace(/[\s-]/g, "");
  }

  // Each rule returns an error message, or "" when the field is valid
  var rules = {
    "full-name": function () {
      var v = value("full-name");
      if (!v) return "Please enter the patient's full name.";
      if (!NAME_PATTERN.test(v)) return "Use letters only (minimum 3 characters).";
      return "";
    },

    "date-of-birth": function () {
      var v = value("date-of-birth");
      if (!v) return "Please select the date of birth.";

      var dob = new Date(v);
      var today = new Date();
      today.setHours(0, 0, 0, 0);

      if (dob > today) return "Date of birth cannot be in the future.";
      if (today.getFullYear() - dob.getFullYear() > 120) return "Please enter a valid date of birth.";
      return "";
    },

    "gender": function () {
      if (!value("gender")) return "Please select a gender.";
      return "";
    },

    "mobile": function () {
      var v = cleanPhone(value("mobile"));
      if (!v) return "Please enter the mobile number.";
      if (!PHONE_PATTERN.test(v)) return "Enter a valid 10-digit Indian mobile number.";
      return "";
    },

    "email": function () {
      var v = value("email");
      if (v && !EMAIL_PATTERN.test(v)) return "Enter a valid email address.";
      return "";
    },

    "address": function () {
      var v = value("address");
      if (!v) return "Please enter the residential address.";
```

```
      if (v.length < 10) return "Address looks too short. Add street, area, city and PIN code.";
      return "";
    },

    "emergency-name": function () {
      var v = value("emergency-name");
      if (!v) return "Please enter the emergency contact name.";
      if (!NAME_PATTERN.test(v)) return "Use letters only (minimum 3 characters).";
      return "";
    },

    "emergency-phone": function () {
      var v = cleanPhone(value("emergency-phone"));
      if (!v) return "Please enter the emergency contact number.";
      if (!PHONE_PATTERN.test(v)) return "Enter a valid 10-digit Indian mobile number.";
      if (v.slice(-10) === cleanPhone(value("mobile")).slice(-10)) {
        return "Emergency number must be different from the patient's number.";
      }
      return "";
    },

    "relationship": function () {
      var v = value("relationship");
      if (!v) return "Please enter the relationship.";
      if (!/^[A-Za-z][A-Za-z ]{1,29}$/.test(v)) return "Use letters only, for example Parent or Spouse.";
      return "";
    },

    "medical-notes": function () {
      if (value("medical-notes").length > 500) return "Please keep the notes under 500 characters.";
      return "";
    },

    "insurance-provider": function () {
      if (value("policy-number") && !value("insurance-provider")) {
        return "Enter the insurance provider name.";
      }
      return "";
    },

    "policy-number": function () {
      var v = value("policy-number");
      if (value("insurance-provider") && !v) return "Enter the policy or member ID.";
      if (v && !POLICY_PATTERN.test(v)) return "Use 4-30 letters, numbers, / or - only.";
      return "";
    },

    "consent": function () {
      if (!document.getElementById("consent").checked) return "Please confirm the consent to continue.";
      return "";
    }
  };

  function getErrorBox(input) {
    var holder = input.closest(".field") || input.closest(".check-row") || input.parentNode;
    var box = holder.querySelector(".error-message");

    if (!box) {
      box = document.createElement("span");
      box.className = "error-message";
      box.setAttribute("role", "alert");
      holder.appendChild(box);
    }

    return box;
  }

  function showError(input, message) {
    var box = getErrorBox(input);
    box.textContent = message;
    input.classList.toggle("field-invalid", Boolean(message));
    input.setAttribute("aria-invalid", message ? "true" : "false");
  }

  function validateField(id) {
    var input = document.getElementById(id);
    var message = rules[id]();
    showError(input, message);
    return message === "";
  }

  // Live feedback while the user fills the form
  Object.keys(rules).forEach(function (id) {
    var input = document.getElementById(id);
    if (!input) return;

```

```
    input.addEventListener("blur", function () {
      validateField(id);
    });

    input.addEventListener("input", function () {
      if (input.classList.contains("field-invalid")) {
        validateField(id);
      }
    });

    input.addEventListener("change", function () {
      validateField(id);
    });
  });

  // ===== DATABASE INTEGRATION: send the form to the server =====

  // JSON key sent to the server -> form field id
  var FIELD_MAP = {
    fullName: "full-name",
    dateOfBirth: "date-of-birth",
    gender: "gender",
    bloodGroup: "blood-group",
    mobile: "mobile",
    email: "email",
    address: "address",
    emergencyName: "emergency-name",
    emergencyPhone: "emergency-phone",
    relationship: "relationship",
    department: "department",
    medicalNotes: "medical-notes",
    insuranceProvider: "insurance-provider",
    policyNumber: "policy-number"
  };

  function collectFormData() {
    var data = {};

    Object.keys(FIELD_MAP).forEach(function (key) {
      data[key] = value(FIELD_MAP[key]);
    });

    data.consent = document.getElementById("consent").checked;
    return data;
  }

  function showFormMessage(message) {
    var box = form.querySelector(".form-message");

    if (!box) {
      box = document.createElement("p");
      box.className = "form-message error-message";
      box.setAttribute("role", "alert");
      form.querySelector(".form-footer").before(box);
    }

    box.textContent = message;
  }

  // Validate everything on submit, then save to the database
  form.addEventListener("submit", async function (event) {
    event.preventDefault();

    var firstInvalid = null;

    Object.keys(rules).forEach(function (id) {
      if (!validateField(id) && !firstInvalid) {
        firstInvalid = document.getElementById(id);
      }
    });

    if (firstInvalid) {
      firstInvalid.focus();
      return;
    }

    var submitButton = form.querySelector("button[type='submit']");
    submitButton.disabled = true;
    showFormMessage("");

    try {
      var response = await fetch("/api/register", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(collectFormData())
      });
```

```
251
252       var result = await response.json();
253
254       if (!response.ok || !result.success) {
255         // Show the errors sent back by the server (for example a duplicate mobile number)
256         Object.keys(result.errors || {}).forEach(function (key) {
257           if (FIELD_MAP[key]) {
258             showError(document.getElementById(FIELD_MAP[key]), result.errors[key]);
259           }
260         });
261
262         showFormMessage(result.message || "Registration failed. Please try again.");
263         return;
264       }
265
266       // Saved in the database
267       window.location.href = "registration-success.html";
268     } catch (error) {
269       showFormMessage("Unable to connect to the server.");
270     } finally {
271       submitButton.disabled = false;
272     }
273   });
274 });
```

Server Source Code – server.js

Node.js server that opens the SQLite database, validates the data again on the server, saves it with a prepared INSERT statement and returns a JSON reply to the browser.

```
// St Joseph Hospital - Patient Registration Server
// Node.js 22.13+ only. No npm install needed.
// Start with:  node server.js   then open http://localhost:3000

const http = require("node:http");
const fs = require("node:fs");
const path = require("node:path");
const { DatabaseSync } = require("node:sqlite");

const PORT = process.env.PORT || 3000;
const PUBLIC_DIR = path.join(__dirname, "public");

// ---------- Database ----------
const db = new DatabaseSync(path.join(__dirname, "hospital.db"));
db.exec(fs.readFileSync(path.join(__dirname, "schema.sql"), "utf8"));

const insertPatient = db.prepare(`
  INSERT INTO patients (
    full_name, date_of_birth, gender, blood_group, mobile, email, address,
    emergency_name, emergency_phone, relationship, department, medical_notes,
    insurance_provider, policy_number, consent
  ) VALUES (
    :fullName, :dateOfBirth, :gender, :bloodGroup, :mobile, :email, :address,
    :emergencyName, :emergencyPhone, :relationship, :department, :medicalNotes,
    :insuranceProvider, :policyNumber, :consent
  )
`);

// ---------- Validation (same rules as validation.js) ----------
const NAME_PATTERN = /^[A-Za-z][A-Za-z .'-]{2,59}$/;
const PHONE_PATTERN = /^(?:\+91|91|0)?[6-9]\d{9}$/;
const EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
const POLICY_PATTERN = /^[A-Za-z0-9\/-]{4,30}$/;
const GENDERS = ["Female", "Male", "Non-binary", "Prefer not to say"];

function text(value) {
  return typeof value === "string" ? value.trim() : "";
}

function phone(value) {
  const digits = text(value).replace(/[\s-]/g, "");
  return PHONE_PATTERN.test(digits) ? "+91" + digits.slice(-10) : "";
}

function validate(body) {
  const errors = {};
  const dob = new Date(text(body.dateOfBirth));

  if (!NAME_PATTERN.test(text(body.fullName))) errors.fullName = "Enter a valid full name.";
  if (isNaN(dob) || dob > new Date()) errors.dateOfBirth = "Enter a valid date of birth.";
  if (!GENDERS.includes(text(body.gender))) errors.gender = "Please select a gender.";
  if (!phone(body.mobile)) errors.mobile = "Enter a valid 10-digit mobile number.";
  if (text(body.email) && !EMAIL_PATTERN.test(text(body.email))) errors.email = "Enter a valid email address.";
  if (text(body.address).length < 10) errors.address = "Address looks too short.";
  if (!NAME_PATTERN.test(text(body.emergencyName))) errors.emergencyName = "Enter a valid contact name.";
  if (!phone(body.emergencyPhone)) errors.emergencyPhone = "Enter a valid 10-digit mobile number.";
  else if (phone(body.emergencyPhone) === phone(body.mobile)) {
    errors.emergencyPhone = "Emergency number must be different from the patient's number.";
  }
  if (!/^[A-Za-z][A-Za-z ]{1,29}$/.test(text(body.relationship))) errors.relationship = "Enter a valid relationship.";
  if (text(body.medicalNotes).length > 500) errors.medicalNotes = "Please keep the notes under 500 characters.";
  if (text(body.policyNumber) && !POLICY_PATTERN.test(text(body.policyNumber))) {
    errors.policyNumber = "Enter a valid policy or member ID.";
  }
  if (body.consent !== true) errors.consent = "Consent is required.";

  return errors;
}

// ---------- Helpers ----------
function sendJson(res, status, data) {
  res.writeHead(status, { "Content-Type": "application/json" });
  res.end(JSON.stringify(data));
}

function readJson(req) {
  return new Promise((resolve, reject) => {
    let raw = "";
```

```
    req.on("data", (chunk) => {
      raw += chunk;
      if (raw.length > 50000) {
        reject(new Error("Request too large"));
        req.destroy();
      }
    });
    req.on("end", () => {
      try {
        resolve(JSON.parse(raw || "{}"));
      } catch (error) {
        reject(error);
      }
    });
  });
}

const MIME = {
  ".html": "text/html; charset=utf-8",
  ".js": "text/javascript; charset=utf-8",
  ".css": "text/css; charset=utf-8",
  ".png": "image/png",
  ".svg": "image/svg+xml"
};

function serveFile(req, res) {
  const urlPath = req.url.split("?")[0];
  const requested = urlPath === "/" ? "/index.html" : decodeURIComponent(urlPath);
  const filePath = path.normalize(path.join(PUBLIC_DIR, requested));

  // Block paths such as /../server.js
  if (!filePath.startsWith(PUBLIC_DIR)) {
    res.writeHead(403);
    return res.end("Forbidden");
  }

  fs.readFile(filePath, (error, content) => {
    if (error) {
      res.writeHead(404, { "Content-Type": "text/plain" });
      return res.end("Not found");
    }
    res.writeHead(200, { "Content-Type": MIME[path.extname(filePath)] || "application/octet-stream" });
    res.end(content);
  });
}

// ---------- Registration API ----------
async function registerPatient(req, res) {
  let body;

  try {
    body = await readJson(req);
  } catch (error) {
    return sendJson(res, 400, { success: false, message: "Invalid request." });
  }

  const errors = validate(body);

  if (Object.keys(errors).length > 0) {
    return sendJson(res, 400, { success: false, message: "Please correct the highlighted fields.", errors });
  }

  try {
    const result = insertPatient.run({
      fullName: text(body.fullName),
      dateOfBirth: text(body.dateOfBirth),
      gender: text(body.gender),
      bloodGroup: text(body.bloodGroup) || null,
      mobile: phone(body.mobile),
      email: text(body.email) || null,
      address: text(body.address),
      emergencyName: text(body.emergencyName),
      emergencyPhone: phone(body.emergencyPhone),
      relationship: text(body.relationship),
      department: text(body.department) || null,
      medicalNotes: text(body.medicalNotes) || null,
      insuranceProvider: text(body.insuranceProvider) || null,
      policyNumber: text(body.policyNumber) || null,
      consent: 1
    });

    return sendJson(res, 201, {
      success: true,
      message: "Registered successfully.",
      patientId: Number(result.lastInsertRowid)
    });
```

```
  } catch (error) {
    if (String(error.message).includes("UNIQUE")) {
      return sendJson(res, 409, {
        success: false,
        message: "This mobile number is already registered.",
        errors: { mobile: "This mobile number is already registered." }
      });
    }

    console.error(error);
    return sendJson(res, 500, { success: false, message: "Server error. Please try again." });
  }
}

// ---------- Server ----------
const server = http.createServer((req, res) => {
  if (req.method === "POST" && req.url === "/api/register") {
    return registerPatient(req, res);
  }

  if (req.method === "GET") {
    return serveFile(req, res);
  }

  res.writeHead(405);
  res.end("Method not allowed");
});

server.listen(PORT, () => {
  console.log("St Joseph Hospital server running at http://localhost:" + PORT);
});
```

SQL Source Code – schema.sql

Creates the patients table that stores every registration. The UNIQUE mobile column prevents duplicate registrations.

```
-- St Joseph Hospital - Patient Registration Database
-- Run once. The server also runs this file automatically at start-up.

CREATE TABLE IF NOT EXISTS patients (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name           TEXT    NOT NULL,
    date_of_birth       TEXT    NOT NULL,
    gender              TEXT    NOT NULL
                        CHECK (gender IN ('Female', 'Male', 'Non-binary', 'Prefer not to say')),
    blood_group         TEXT,
    mobile              TEXT    NOT NULL UNIQUE,
    email               TEXT,
    address             TEXT    NOT NULL,
    emergency_name      TEXT    NOT NULL,
    emergency_phone     TEXT    NOT NULL,
    relationship        TEXT    NOT NULL,
    department          TEXT,
    medical_notes       TEXT,
    insurance_provider  TEXT,
    policy_number       TEXT,
    consent             INTEGER NOT NULL CHECK (consent = 1),
    registered_at       TEXT    NOT NULL DEFAULT (datetime('now', 'localtime'))
);

CREATE INDEX IF NOT EXISTS idx_patients_name ON patients (full_name);
```

HTML Source Code – registration-success.html

The page shown after the registration is saved, with the confirmation message, next-steps panel and a back-to-login button.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Registration confirmation page for St Joseph Hospital.">
  <title>St Joseph Hospital | Registration Successful</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
      padding: 30px 20px;
      font-family: "Segoe UI", Arial, Helvetica, sans-serif;
      background: linear-gradient(135deg, #e8edff, #ffffff);
      color: #172033;
    }

    .success-card {
      width: 460px;
      max-width: 100%;
      padding: 42px 40px 36px;
      text-align: center;
      background: #ffffff;
      border: 1px solid #e2e5ec;
      border-radius: 20px;
      box-shadow: 0 20px 50px rgba(0, 0, 0, 0.12);
    }

    .brand-mark {
      width: 56px;
      height: 56px;
      margin: 0 auto 10px;
    }

    .brand-mark svg {
      width: 100%;
      height: 100%;
    }

    .eyebrow {
      font-size: 12px;
      font-weight: 700;
      letter-spacing: 0.18em;
      color: #315be8;
      margin-bottom: 26px;
    }

    .check-circle {
      width: 92px;
      height: 92px;
      margin: 0 auto 22px;
      display: flex;
      align-items: center;
      justify-content: center;
      border-radius: 50%;
      background: #e7f7ee;
      box-shadow: 0 0 0 10px #f3fbf6;
    }

    .check-circle svg {
      width: 48px;
      height: 48px;
    }

    h1 {
      font-size: 30px;
      margin-bottom: 10px;
    }

    .message {
```

```
      font-size: 14.5px;
      line-height: 1.6;
      color: #475467;
      margin-bottom: 26px;
    }

    .next-steps {
      text-align: left;
      padding: 18px 20px;
      margin-bottom: 28px;
      background: #f5f7ff;
      border: 1px solid #dfe5fb;
      border-radius: 12px;
    }

    .next-steps h2 {
      font-size: 13px;
      letter-spacing: 0.06em;
      text-transform: uppercase;
      color: #315be8;
      margin-bottom: 10px;
    }

    .next-steps ul {
      list-style: none;
    }

    .next-steps li {
      position: relative;
      padding-left: 22px;
      margin-bottom: 8px;
      font-size: 13.5px;
      color: #344054;
    }

    .next-steps li:last-child {
      margin-bottom: 0;
    }

    .next-steps li::before {
      content: "\2713";
      position: absolute;
      left: 0;
      font-weight: 700;
      color: #12a150;
    }

    .btn {
      display: block;
      width: 100%;
      height: 48px;
      line-height: 48px;
      border-radius: 8px;
      background: #315be8;
      color: #ffffff;
      font-size: 15px;
      font-weight: 700;
      text-decoration: none;
      transition: 0.3s;
    }

    .btn:hover {
      background: #2448c5;
      transform: translateY(-2px);
      box-shadow: 0 8px 20px rgba(49, 91, 234, 0.25);
    }

    .footnote {
      margin-top: 18px;
      font-size: 12.5px;
      color: #667085;
    }

    @media (max-width: 600px) {
      .success-card {
        padding: 32px 22px 28px;
        border-radius: 16px;
      }

      h1 {
        font-size: 26px;
      }
    }
  </style>
</head>
<body>
```

```
  <main class="success-card" aria-labelledby="success-title">
    <div class="brand-mark" aria-hidden="true">
      <svg viewBox="0 0 64 64">
        <circle cx="32" cy="32" r="30" fill="#eaf0ff"/>
        <rect x="27" y="14" width="10" height="36" rx="3" fill="#315be8"/>
        <rect x="14" y="27" width="36" height="10" rx="3" fill="#315be8"/>
      </svg>
    </div>
    <p class="eyebrow">ST JOSEPH HOSPITAL</p>

    <div class="check-circle" aria-hidden="true">
      <svg viewBox="0 0 48 48" fill="none" stroke="#12a150" stroke-width="5" stroke-linecap="round" stroke-linejoin="round">
        <path d="M10 25 L20 35 L38 14"/>
      </svg>
    </div>

    <h1 id="success-title">Registered Successfully!</h1>
    <p class="message">
      Your patient registration has been submitted successfully.
      Our team will review your details and contact you shortly.
    </p>

    <section class="next-steps" aria-label="What happens next">
      <h2>What happens next</h2>
      <ul>
        <li>Your details are saved securely and kept confidential.</li>
        <li>Carry a valid photo ID and insurance card to your visit.</li>
        <li>Arrive 15 minutes early for your first consultation.</li>
      </ul>
    </section>

    <a class="btn" href="login.html">Back to Login &rarr;</a>
    <p class="footnote">Thank you for choosing St Joseph Hospital.</p>
  </main>
</body>
</html>
```